

Sideband spec document

UCle 3.0 Physical Layer

Version 1

Block Owner

Authors

Abdelrahman Mohamed Ragab

Omar Mohamed Araby

Youssef Gamal Eldein

This Page is left Blank Intentionally

1 Table of Contents

1	Table of Contents	1
2	Operation and Description.....	2
2.1	Digital Interface	2
2.1.1	Parameters Names.....	2
2.1.2	Ports Names	2
3	Functional Description	4
3.1	Block Overview and Theory of Operation.....	4
3.2	Functional Specification: LTSM-to-Link (Transmit Path).....	6
3.2.1	Overview	6
3.2.2	Hardware Interfaces	6
3.2.3	Functional Blocks & Data Flow.....	6
3.2.3.1	Message Encoding (u_tx_msg_enc_dec & u_rx_msg_enc_dec).....	6
3.2.3.2	Input Buffering (u_traffic_tx_fifo & u_traffic_rx_fifo)	6
3.2.3.3	Arbitration (ucie_sb_traffic)	7
3.2.3.4	Serialization and Transmission	7
3.2.4	Message Construction & Formatting.....	7
3.3	Functional Specification: Link-to-LTSM (Receive Path)	8
3.3.1	Overview	8
3.3.2	Hardware Interfaces	8
3.3.3	Functional Blocks & Data Flow.....	9
3.4	Sideband Packet Formats and Physical Transmission.....	10
3.4.1	Packet Types and Identifiers	10
3.4.2	Message Formats	10
3.4.3	Sideband Signaling	11
3.4.4	Serial Transmission Protocol.....	11
4	Block Diagram	12
5	Timing Diagrams	14
6	Verification Requirements	17

2 Operation and Description

2.1 Digital Interface

2.1.1 Parameters Names

Parameter Name	Default	Local	Description
pMSG_WIDTH	128	No	Width of message bus
pSER_WIDTH	64	No	Width of pre_serialized output
pFIFO_WIDTH	128	No	Width of FIFO element
pFIFO_DEPTH	32	No	Depth of the FIFO memory
pENCODING_WIDTH	9	No	Width of Encoding field
pDECODING_WIDTH	9	No	Width of Decoding field
pDATA_WIDTH	64	No	Width of data field
pINFO_WIDTH	16	No	Width of info field
pMSG_CODE_WIDTH	8	No	Width of message code field
pMSG_SUBCODE_WIDTH	8	No	Width of message subcode field
pOP_CODE_WIDTH	5	No	Width of operation code field
pRESERVED	0	No	Reserved bits
lp_max_fifo_addr	$\$clog2(pFIFO_DEPTH)$	Yes	Max of FIFO Address

2.1.2 Ports Names

Port Name	Port Width	Port Type	Description
i_clk	1	input	The system clock
i_reset	1	input	The global reset
i_tx_sb_req	1	input	Indicate Req message is Sent from Tx to SB
i_tx_sb_rsp	1	input	Indicate Rsp message is Sent from Tx to SB
i_rx_sb_req	1	input	Indicate Req message is Sent from Rx to SB
i_rx_sb_rsp	1	input	Indicate Rsp message is Sent from Rx to SB
i_tx_sb_done	1	input	Indicate that Tx Take the message from SB
i_rx_sb_done	1	input	Indicate that Rx Take the message from SB
i_tx_encoding	9	input	Encoding of the incoming message from Tx
i_rx_encoding	9	input	Encoding of the incoming message from Rx

i_tx_info	16	input	The info field of incoming message from Tx
i_rx_info	16	input	The info field of incoming message from Rx
i_tx_data	64	input	The data of the incoming message from Tx
i_rx_data	64	input	The data of the incoming message from Rx
i_t1_ms	1	input	Indication of passing 1ms
i_sb_init_start	1	input	Indication the start of the SB init state and it maintain to be 1 in it
i_rx_sb_data	1	input	The incoming data to the SB from the other UCle module
i_rx_sb_clk	1	input	The incoming clock to the SB from the other UCle module
o_tx_decoding	9	output	Decoding of the message being sent to Tx
o_rx_decoding	9	output	Decoding of the message being sent to Rx
o_tx_info	16	output	The info field of the message being sent to Tx
o_rx_info	16	output	The info field of the message being sent to Rx
o_tx_data	64	output	The data of the message being sent to Tx
o_rx_data	64	output	The data of the message being sent to Rx
o_tx_valid	1	output	Indicate to LTSM Tx that the message is valid (parity check)
o_rx_valid	1	output	Indicate to LTSM Rx that the message is valid (parity check)
o_sb_tx_req	1	output	Indicate Req message is Sent from SB to Tx
o_sb_tx_rsp	1	output	Indicate Rsp message is Sent from SB to Tx
o_sb_rx_req	1	output	Indicate Req message is Sent from SB to Rx
o_sb_rx_rsp	1	output	Indicate Rsp message is Sent from SB to Rx
o_sb_tx_done	1	output	Indicate that SB Take the message from Tx
o_sb_rx_done	1	output	Indicate that SB Take the message from Rx
o_sb_ready	1	output	Indicate to LTSM that Sideband Is ready
o_tx_sb_data	1	output	The sent data to the other UCle module from the SB
o_tx_sb_clk	1	output	The sent CLK to the other UCle module from the SB

3 Functional Description

3.1 Block Overview and Theory of Operation

The Sideband (SB) block is a low-bandwidth communication interface responsible for link initialization and reliable exchange between UCle modules. It ensures that two silicon dies can coordinate their startup sequence independently of the high-speed data path.

Architecture and Port Connectivity

- **Symmetric Design:** The block utilizes a symmetric TX/RX architecture to manage both outgoing and incoming traffic control.
- **Physical Interface:** The block interfaces with other UCle modules via the `i_rx_sb_data` and `i_rx_sb_clk` input ports, and the `o_tx_sb_data` and `o_tx_sb_clk` output ports.
- **Initialization Engines:** The Pattern Generator and Pattern Detector manage the physical layer handshake by toggling patterns based on the `i_sb_init_start` signal. They inform the LTSM that the SB is ready to be used by asserting the `o_sb_ready` signal after the SB initialization sequence is complete.
- **Message Processing:**
 - **Encoding Path (Inputs):** Receives 9-bit message types (`i_tx_encoding`, `i_rx_encoding`) and 64-bit data payloads (`i_tx_data`, `i_rx_data`) to be formatted for transmission.
 - **Decoding Path (Outputs):** Provides 9-bit decoded message types (`o_tx_decoding`, `o_rx_decoding`) and 64-bit payloads (`o_tx_data`, `o_rx_data`) to the local system.
 - **Handshake Tracking:** Monitors and indicates the current state of handshaking for both received and sent messages from either the TX or RX pathways.

Initialization Flow

1. **Trigger:** The LTSM asserts the `i_sb_init_start` signal to indicate to the Sideband that an initialization sequence is triggered.
2. **Pattern Generation:** The Sideband begins sending iterations of a 64 UI clock pattern (defined as toggling every Unit Interval, e.g., 1010...) followed by 32 UI driven low on its TX data line (`o_tx_sb_data`). Concurrently, the UCle Module must send a strobe on its sideband clock (`o_tx_sb_clk`) during the 64 UI clock pattern duration and hold it low (gated) otherwise.
3. **Pattern Detection:** The Sideband RX must sample incoming data patterns on `i_rx_sb_data` using the incoming clock `i_rx_sb_clk`. Pattern detection is considered successful if a continuous 128 UI clock pattern is detected.
4. **Successful Handshake:** If the Sideband RX successfully detects the pattern, the Sideband TX stops sending data and clock after four additional pattern iterations. It then asserts a stop signal to inform the LTSM that it is ready, allowing for any timing differences between the two UCle Modules coming out of reset. At this point, the UCle Module Sideband TX and RX must be enabled to send and receive sideband messages, respectively.
5. **Timeout and Error Handling:** If the pattern is not detected on the RX side, the Sideband continues to alternate between sending the pattern on its TX for 1 ms and holding it low for 1 ms, up to a total time of 8 ms. The sideband RX must remain enabled during this time. If a

timeout occurs after 8 ms without the stop signal being asserted by the Sideband, the LTSM must transition to the TRAINERROR state and deassert `i_sb_init_start`.

6. **Out of Reset Message:** Once the `o_sb_ready` signal is asserted, the LTSM must continuously transmit the {SBINIT Out of Reset} sideband message on `o_tx_sb_data` (along with `o_tx_sb_clk`) until it detects the identical message on its sideband RX, or until a timeout occurs.
7. **Completion:** Following this sequence, the Sideband operates normally with the LTSM.

Normal Operation Flow

1. **LTSM-to-Sideband Transmission:** When the LTSM TX or RX needs to send a request or response, it asserts `i_tx_sb_req`, `i_tx_sb_rsp`, `i_rx_sb_req`, or `i_rx_sb_rsp`. The Sideband captures the message and asserts `o_sb_tx_done` or `o_sb_rx_done` respectively. Once the 'done' signal is asserted, indicating successful capture, the LTSM must deassert its corresponding request or response signal.
2. **Sideband-to-LTSM Transmission:** When the Sideband needs to send a received request or response to the LTSM, it asserts `o_sb_tx_req`, `o_sb_tx_rsp`, `o_sb_rx_req`, or `o_sb_rx_rsp`. The LTSM acknowledges receipt by asserting `i_tx_sb_done` or `i_rx_sb_done` respectively. Upon this acknowledgment, the Sideband deasserts its request or response signal.
3. **Data and Encoding Capture:** Data and encoding transmissions to the Sideband follow the same handshake mechanism. The TX and RX state machines send message specifics via `i_tx_encoding`, `i_tx_data`, `i_rx_encoding`, and `i_rx_data`. The Sideband utilizes these signals alongside the request/response strobes to construct the message and store it inside the internal TX FIFO or RX FIFO.
4. **Data and Encoding Delivery:** Data and encoding transmissions from the Sideband to the LTSM similarly follow this mechanism. The Sideband decodes messages from the internal FIFOs and drives them onto `o_tx_decoding`, `o_tx_data`, `o_rx_decoding`, and `o_rx_data`. These signals are sent alongside the request/response strobes to provide the complete message information to the local system.

3.2 Functional Specification: LTSM-to-Link (Transmit Path)

3.2.1 Overview

The LTSM-to-Link (Transmit) path is responsible for taking high-level initialization and control directives from the local LTSM (both TX and RX state machines) and translating them into compliant UCle 3.0 Physical Link sideband messages. It ensures that concurrent message requests are safely buffered in hardware FIFOs, fairly arbitrated, correctly formatted with hardware parity, and serialized across the physical `o_tx_sb_data` line aligned with the `o_tx_sb_clk` strobe.

3.2.2 Hardware Interfaces

The LTSM-to-Link path interacts with the local LTSM and the remote physical link using the following key signals:

- **LTSM Request/Response Interfaces:**
 - `i_tx_sb_req / i_tx_sb_rsp`: Indicates a request/response message is being sent from the LTSM TX FSM.
 - `i_rx_sb_req / i_rx_sb_rsp`: Indicates a request/response message is being sent from the LTSM RX FSM.
 - `i_tx_encoding / i_rx_encoding` (9-bit): The specific message encoding type requested by the TX or RX FSM.
 - `i_tx_info / i_rx_info` (16-bit): The informational payload for the message.
 - `i_tx_data / i_rx_data` (64-bit): The data payload for the message.
- **LTSM Acknowledge Interfaces:**
 - `o_sb_tx_done / o_sb_rx_done`: Asserts to indicate the Sideband block has successfully captured the message into its internal FIFOs.
- **Physical Link Interfaces:**
 - `o_tx_sb_data`: The serialized data bitstream sent to the remote UCle module.
 - `o_tx_sb_clk`: The forwarded clock strobe used by the remote receiver to sample `o_tx_sb_data`.

3.2.3 Functional Blocks & Data Flow

3.2.3.1 Message Encoding (`u_tx_msg_enc_dec` & `u_rx_msg_enc_dec`)

When the LTSM asserts a request (`i_tx_sb_req` or `i_rx_sb_req`), the corresponding encoder block captures the 9-bit encoding, 16-bit info, and 64-bit data signals. The encoder's primary function is to translate the 9-bit local encoding into the standard UCle message components (Opcode, MsgCode, MsgSubcode). It performs the parity calculations on-the-fly to construct a flat 128-bit message vector.

3.2.3.2 Input Buffering (`u_traffic_tx_fifo` & `u_traffic_rx_fifo`)

To prevent the LTSM from stalling during back-to-back requests or slow serial transmission, the 128-bit encoded messages are pushed into dedicated hardware FIFOs.

- **Capacity:** Each FIFO is parameterized by `PFIFO_WIDTH` (128 bits) and `PFIFO_DEPTH` (default 32 entries).

- **Handshaking:** Once the encoded message is securely written to the FIFO, the Sideband block asserts `o_sb_tx_done` (or `o_sb_rx_done`). Upon seeing this assertion, the LTSM must deassert its request line.

3.2.3.3 Arbitration (`ucie_sb_traffic`)

Because both the TX and RX state machines of the LTSM can generate sideband messages concurrently, the `ucie_sb_traffic` block acts as the central arbiter.

- It monitors the `empty` flags of both the `u_traffic_tx_fifo` and `u_traffic_rx_fifo`.
- If both FIFOs contain data, it employs a fair round-robin arbitration scheme (alternating between TX and RX messages) to pop data into the serializer pathway. The alternation starts with sending a TX message then an RX message then a TX message and so on till transmitting all the messages existing in the FIFOs.
- If only one FIFO contains data, it continuously drains that FIFO to maximize throughput.

3.2.3.4 Serialization and Transmission

The winning 128-bit message is forwarded `traffic_fifo` block to identify the type of the message then send a 64 bit message to the serializer block (`u_ser`). * The serializer converts the 64-bit parallel data into a serial bitstream.

- The bitstream is driven onto `o_tx_sb_data` at the rate of the 800 MT/s single data rate signal with an 800 MHz source.
- Simultaneously, the block generates the `o_tx_sb_clk` strobe, toggling it for the duration of the message transmission to allow the remote link to sample the data correctly.

3.2.4 Message Construction & Formatting

The encoded 128-bit message is constructed across 4 logical phases (32 bits each) before serialization into 64 bit stobes. The hardware formats it as follows:

- **Phase 0 (Header 1):**
 - Bits [4:0]: opcode (e.g., `10010b` for Message without Data, or `11011b` for Message with 64b Data).
 - Bits: `msgcode[7:0]` (Upper byte of the message code).
 - Bits: `srcid` (Set to `010b` to indicate the Physical Layer).
- **Phase 1 (Header 2):**
 - Bits [7:0]: `MsgSubcode[7:0]` (Lower byte of the message code).
 - Bits [23:8]: `MsgInfo[15:0]`.
 - Bits: `dstid` (Set to `110b` for a remote die terminated request targeting the Physical Layer).
 - Bit [30]: `cp` (Control Parity). The hardware calculates this as the even parity of all header bits excluding the DP bit.
 - Bit [31]: `dp` (Data Parity). The hardware calculates this as the even parity of all bits in the data payload. If there is no data payload (Phase 2 and 3 are empty), this bit is tied to `0b`.
- **Phase 2 & 3 (Data Payload):**
 - Contains the 64-bit data payload if the opcode dictates a message with data (resulting in a 128-bit transmission). If the message is without data, these phases are completely

omitted, and the Sideband transmits only the 64-bit header (Phase 0 and Phase 1). No zero-padding is performed.

3.3 Functional Specification: Link-to-LTSM (Receive Path)

3.3.1 Overview

The Link-to-LTSM (Receive) path ingests the remote die's sideband stream on `i_rx_sb_data` sampled by `i_rx_sb_clk`, reconstructs 64-bit/128-bit messages via deserialization, validates header and (if present) data parity, classifies each message as TX-directed or RX-directed, and delivers the decoded control, info, and data to the local LTSM through the `o_tx_*` / `o_rx_*` interfaces together with the appropriate request strobes and validity indications. The path is active after SB initialization completes and `o_sb_ready` is asserted, and it remains able to accept traffic continuously while honoring FIFO backpressure and LTSM handshakes. (Derived from Functional Description, Deserialization and receiving, and Valid signal and parity sections in this document.)

3.3.2 Hardware Interfaces

Physical link inputs

- `i_rx_sb_data` (1-bit) — Serial data stream from remote UCle module.
- `i_rx_sb_clk` (1-bit) — Source-synchronous strobe used to sample `i_rx_sb_data`. (See Functional Description.)

LTSM message delivery to local TX / RX FSMs

- `o_sb_tx_req` / `o_sb_tx_rsp` (1-bit) — Asserted when a received message targets the TX FSM (request/response).
- `o_sb_rx_req` / `o_sb_rx_rsp` (1-bit) — Asserted when a received message targets the RX FSM (request/response).
- `o_tx_decoding`, `o_rx_decoding` (9-bit) — Decoded message type for TX-directed or RX-directed delivery, respectively.
- `o_tx_info`, `o_rx_info` (16-bit) — Message info field delivered with the header.
- `o_tx_data`, `o_rx_data` (64-bit) — Data payload when present (messages with data).
- `o_tx_valid`, `o_rx_valid` (1-bit) — Indicates header and data parity are correct for the delivered message. If parity fails, the respective valid deasserts.
- `i_tx_sb_done`, `i_rx_sb_done` (1-bit, from LTSM) — Acknowledge that LTSM has consumed the message; the receive path uses these to complete handshakes and deassert its request/response strobes.

3.3.3 Functional Blocks & Data Flow

- Deserializer: Samples `i_rx_sb_data` on `i_rx_sb_clk` and assembles 64-bit words matching the transmitted strobe framing (64 UI run, 32 UI low). Pushes each reassembled 64-bit word into an internal deserializer FIFO for downstream parsing.
- Header/Data Extract & OpCode: Pops one 64-bit word from the deserializer FIFO and inspects the opcode (Phase 0/Phase 1 encapsulated in the 64-bit header). If the opcode indicates message without data, it asserts `msg_ready` and forwards the 64-bit header to the traffic classifier. If the opcode indicates message with data, it pops the next 64-bit word (data) after the required 32-UI low gap, forms the 128-bit message, and forwards it to the traffic classifier. (See Messages and Message Construction & Formatting.)
- Parity Check: Computes Control Parity (CP) as even parity over header bits excluding DP, and Data Parity (DP) as even parity over all payload bits when data is present. Flags a parity failure if either check fails; the downstream delivery will set `o_tx_valid` or `o_rx_valid` low accordingly. Then wait for the timeout to occur (8ms), although its seem insufficient but the bit error rate is very low $BER = 10^{-23}$
- Traffic Classification: Classifies each received message as directed to TX or RX based on the message code conventions: Most Req toward RX have `msg_code[3:0] = 4'h5`; most Rsp toward TX have `msg_code[3:0] = 4'hA`.
- Exceptions are explicitly routed as:
 - To TX: `msg_code = 8'h85`, `msg_subcode = 8'h0A`
 - To RX: `msg_code = 8'h8A`, `msg_subcode = 8'h0A`
 - To RX: `msg_code = 8'h91`, `msg_subcode = 8'h00`
 - To RX: `msg_code = 8'h81`, `msg_subcode = 8'h0C`
- Per-Destination Delivery & Handshake:
 - TX-directed messages: drives `o_sb_tx_req` or `o_sb_tx_rsp` with `o_tx_decoding`, `o_tx_info`, and `o_tx_data` (if present). Validity is indicated on `o_tx_valid`.
 - RX-directed messages: drives `o_sb_rx_req` or `o_sb_rx_rsp` with `o_rx_decoding`, `o_rx_info`, and `o_rx_data` (if present). Validity is indicated on `o_rx_valid`.
 - Holds the corresponding request/response strobe asserted until LTSM returns `i_tx_sb_done` or `i_rx_sb_done`, then deasserts the strobe to complete the handshake.

3.4 Sideband Packet Formats and Physical Transmission

This section details the physical signaling, structural formatting, and transmission protocols of the Sideband packets.

3.4.1 Packet Types and Identifiers

Every sideband packet type carries a 5-bit opcode, a 3-bit source identifier (*srcid*), and a 3-bit destination identifier (*dstid*).

Based on the current architecture, the Sideband block implements a specific subset of the UCle standard:

- **Supported Opcodes:** The design currently supports only two packet types:
- **10010b:** Message without Data.
- **11011b:** Message with 64b Data.
- **Supported Identifiers:** The design currently interfaces exclusively with the LTSM TX and RX pathways; the RDI interface is not yet supported. Furthermore, UCle Management is not supported by this block. Therefore, the block utilizes fixed routing identifiers:
- *srcid* is restricted to 010b (Physical Layer).
- *dstid* is restricted to 110b (Physical Layer message, remote die terminated request).

3.4.2 Message Formats

While sideband packets are transported over the physical bumps as 64-bit serial packets, their structural format is conceptually defined over multiple 32-bit Phases.

- **Messages without Data:** These messages consist solely of a 64-bit header, composed of Phase 0 and Phase 1. They do not carry an additional data payload.

Messages without Data																																	
Bytes	3								2								1								0								
Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Header / Data	Header																																
Phase0	srcid			rsvd			rsvd			msgcode[7:0]								rsvd								opcode[4:0]							
Phase1	dp	cp	rsvd			dstid			MsgInfo[15:0]																MsgSubcode[7:0]								

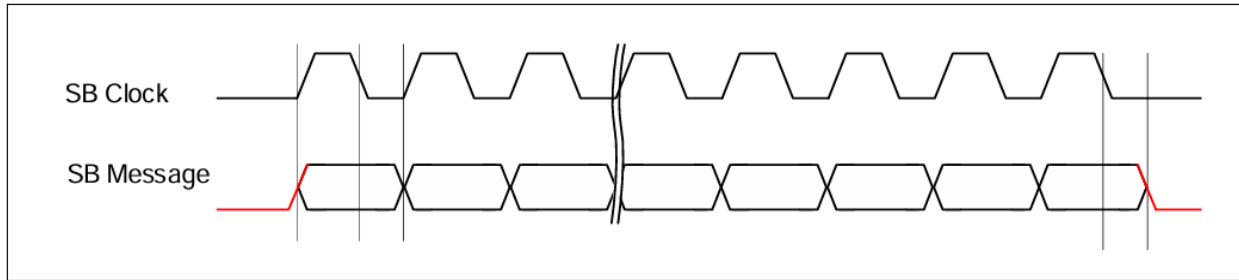
- **Messages with 64b Data:** These messages consist of the standard 64-bit header (Phase 0 and Phase 1) followed immediately by a 64-bit data payload, representing Phase 2 and Phase 3.

Messages with data																																
Bytes	3								2								1								0							
Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Header / Data	Header																															
Phase0	srcid			rsvd			rsvd			msgcode[7:0]								rsvd								opcode[4:0]						
Phase1	dp	cp	rsvd			dstid			MsgInfo[15:0]																MsgSubcode[7:0]							
Header / Data	Data																															
Phase2	data[31:0]																															
Phase3	data[63:32]																															

3.4.3 Sideband Signaling

The sideband utilizes a two-signal physical interface (data and clock) for both transmit and receive directions.

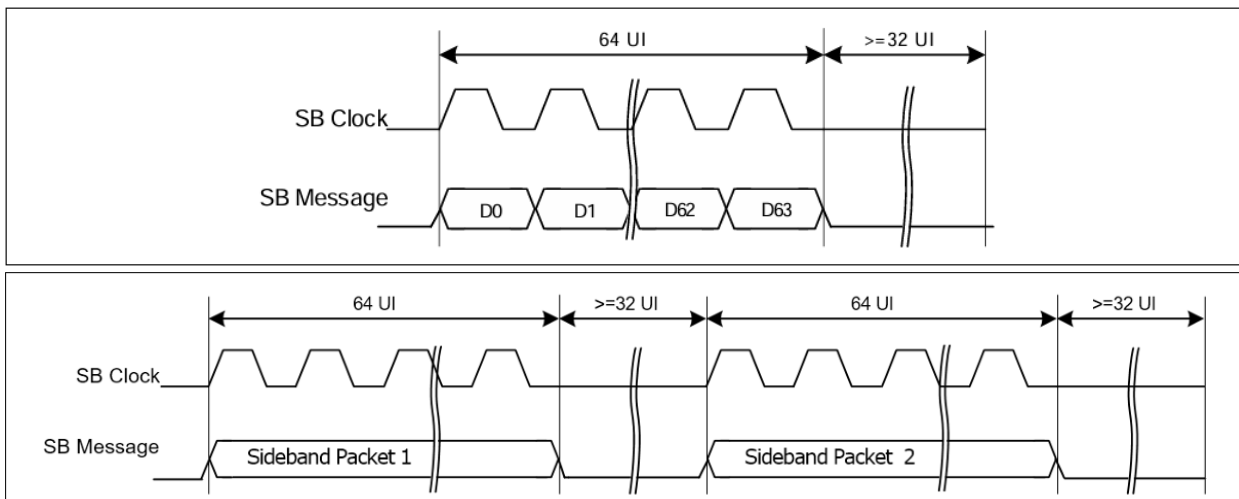
- **Data Rate & Clock:** The interface operates as an 800 MT/s Single Data Rate (SDR) signal, driven by an 800-MHz source clock.
- **Alignment and Sampling:** Sideband data is transmitted edge-aligned with the positive edge of the strobe (clock). The receiving module must sample the incoming data using the negative edge of the strobe.



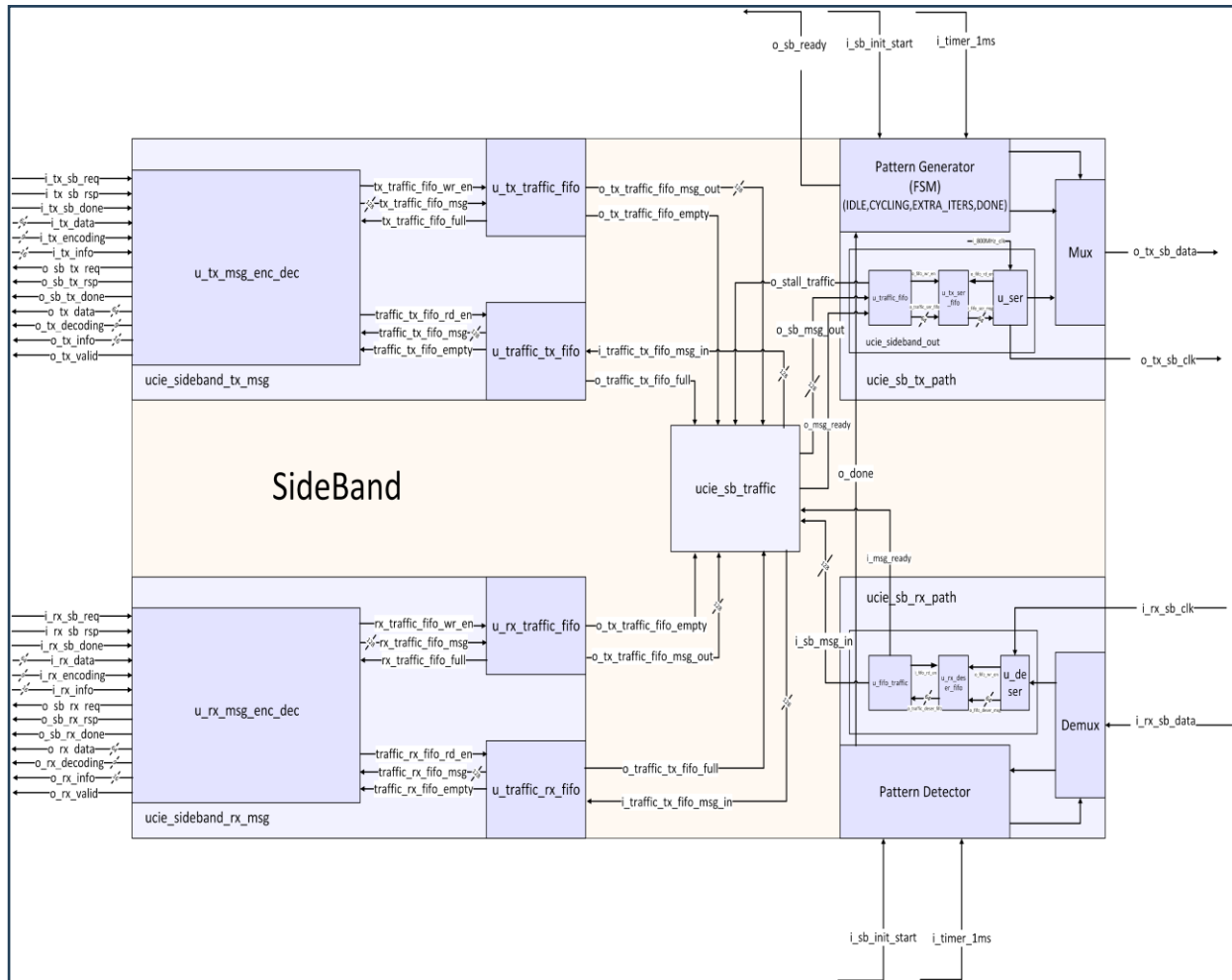
3.4.4 Serial Transmission Protocol

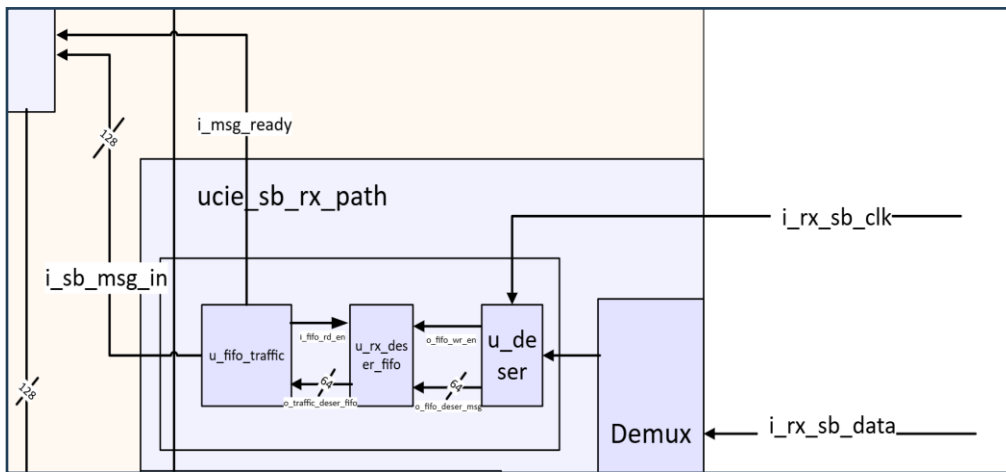
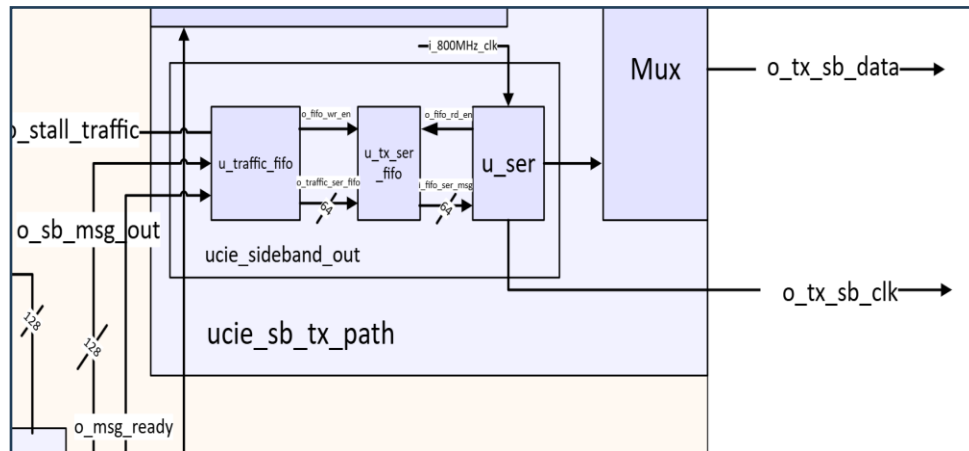
To transport the multi-phase messages over the serial interface, the sideband block adheres to strict serialization and spacing rules.

- **Serialization Order:** The transfer occurs one 64-bit serial packet at a time. For the header, transmission begins with bit 0 of Phase 0 as the first bit on the serial line, proceeding sequentially up to bit 31 of Phase 1 as the 64th bit.
- **Data Payload Transmission:** If the message contains data, it is sent as a subsequent 64-bit serial packet. The order starts with bit 0 of Phase 2 and proceeds to bit 31 of Phase 3.
- **Packet Spacing (Back-to-Back):** Any two 64-bit sideband serial packets on the interface must be separated by a minimum of 32 bits driven low (≥ 32 UI).
- **Full Message Profile:** Consequently, a message containing data is transferred as a 64-bit header, followed by a minimum of 32 bits low, followed by the 64-bit data packet, followed by a minimum of 32 bits low.



4 Block Diagram



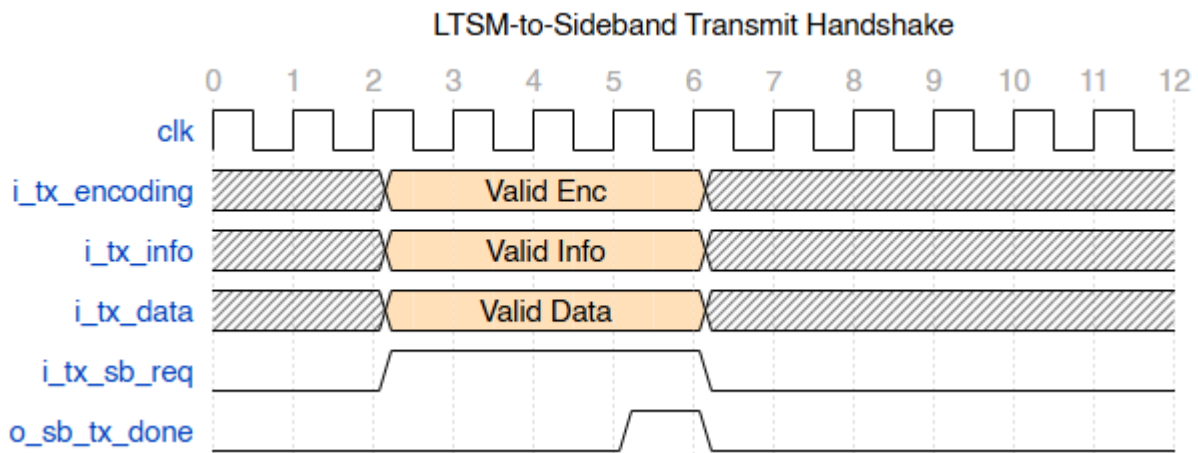


5 Timing Diagrams

LTSM-to-Sideband Transmit Handshake

This diagram illustrates the standard request-and-acknowledge protocol used when the local LTSM transmits a message to the Sideband block.

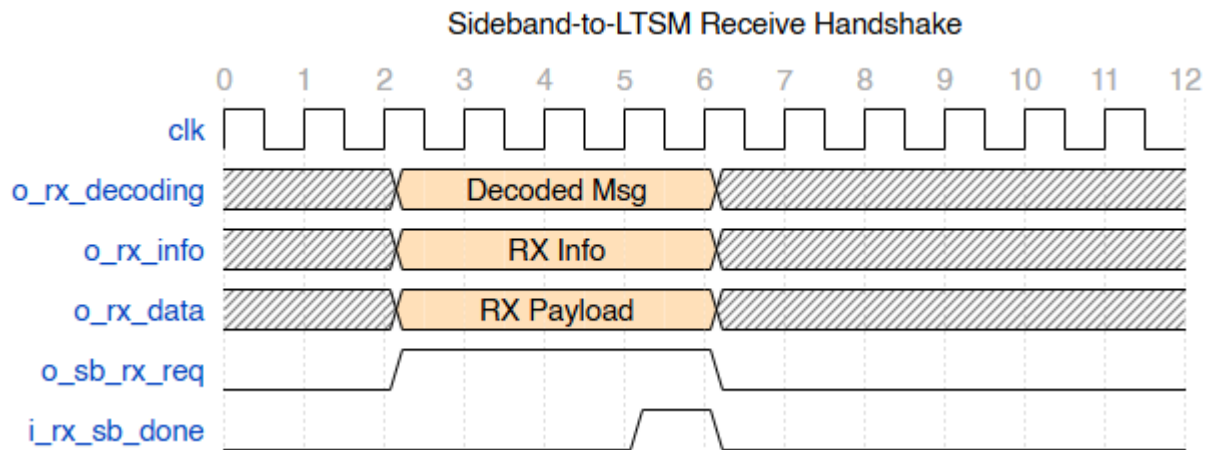
1. The LTSM first drives valid message parameters onto the `i_tx_encoding`, `i_tx_info`, and `i_tx_data` buses.
2. Once the data is stable, the LTSM asserts `i_tx_sb_req` (or `i_tx_sb_rsp`).
3. The Sideband block captures this data into its internal TX FIFO and subsequently asserts `o_sb_tx_done` to signal a successful capture.
4. Upon sampling `o_sb_tx_done` high, the LTSM deasserts its request signal.



Sideband-to-LTSM Receive Handshake

This waveform details the synchronous handshake used when the Sideband block transfers a decoded incoming message to the local LTSM.

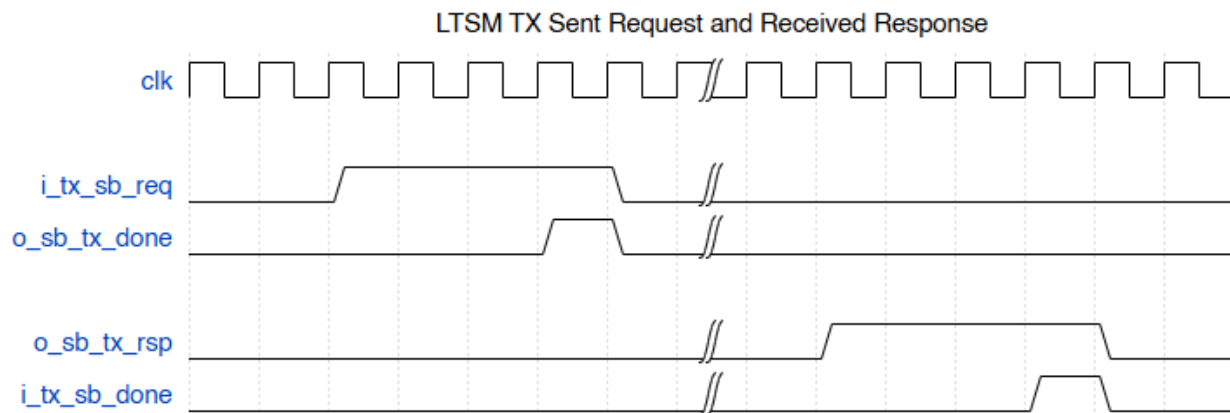
1. The Sideband presents the stable, decoded message parameters on the `o_rx_decoding`, `o_rx_info`, and `o_rx_data` output buses.
2. It then asserts `o_sb_rx_req` (or `o_sb_rx_rsp`) to alert the LTSM.
3. The LTSM captures the presented data and asserts `i_rx_sb_done` to acknowledge receipt.
4. The Sideband detects this acknowledgment, deasserts its request line.



LTSM TX Request and Subsequent Response

This diagram depicts a complete transaction cycle where the LTSM initiates a request and later receives a corresponding response from the Sideband.

1. The transaction begins with the standard LTSM-to-Sideband handshake, where `i_tx_sb_req` is asserted and acknowledged by `o_sb_tx_done`.
2. Following an indeterminate latency period (|), representing the time required for physical transmission and remote processing.
3. The Sideband asserts `o_sb_tx_rsp`, transferring the response data back to the LTSM, which acknowledges the transfer by asserting `i_tx_sb_done`.



6 Verification Requirements

1. Verify that SB manages the initialization sequence successfully.
2. Verify that timeout occurred initialization.
3. Verify that Normal transactions of Req and Rsp between SB and TX, RX is done successfully.
4. Stall and FIFO cases in traffic module.
5. Ser/Deser operation.
6. Full flow of normal operation Tx/Rx.
7. Valid and Invalid messages